Übung 5: Binäre Suchbäume

(Dauer: 90 Minuten - Gruppenarbeit empfohlen)

Ziel der Übung: Sie lernen, binäre Suchbäume und AVL-Bäume zu konstruieren und zu balancieren sowie die Laufzeit grundlegender Operationen wie Suchen und Einfügen asymptotisch zu analysieren.

1. Für einen binären Baum mit maximaler Knotenzahl gilt

$$h = \lceil \log_2(n+1) \rceil - 1.$$

   Bestimmen Sie, wie sich die maximale Anzahl der Knoten verändert, wenn die Höhe h um 1 erhöht wird. Geben Sie die Anzahl der zusätzlichen Knoten in Abhängigkeit von h an.

2. Bestimmen Sie die maximale Anzahl von Knoten in einem binären Baum der Höhe h durch eine kombinatorische Überlegung.
   (a) Bestimmen Sie zunächst die maximale Anzahl von Knoten auf jeder Ebene des Baumes.
   (b) Summieren Sie anschließend die Knotenzahlen aller Ebenen und geben Sie das Ergebnis als geschlossene Formel in Abhängigkeit von h an.

3. Fügen Sie die folgenden Schlüssel in der jeweils angegebenen Reihenfolge in einen zunächst leeren AVL-Baum ein. Zeichnen Sie nach jeder notwendigen Rotation den resultierenden Baum.
   (a) {“penguin”, “stork”, “cat”, “fowl”, “moth”, “badger”, “otter”, “shrew”, “lion”, “raven”, “bat”}
   (b) $\{6, 43, 7, 42, 59, 63, 11, 21, 56, 54, 27, 20, 36\}$

4. Fügen Sie nacheinander die Schlüssel 52, 95 und 71 in den folgenden AVL-Baum ein. Stellen Sie nach jedem Einfügen die AVL-Eigenschaft durch geeignete Rotationen wieder her und zeichnen Sie die jeweiligen Zwischenschritte.

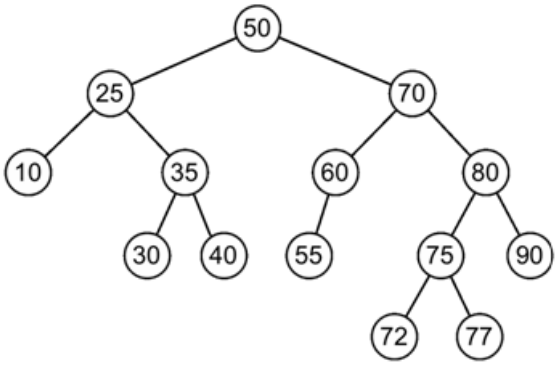


Abbildung 1: AVL-Baum.

5. Betrachten Sie die beiden folgenden Methoden `test` und `populate`.

```
public int test(int n) {
  IDictionary<Integer, Integer> dict = new AvlDictionary<>();
  populate(n, dict);
  int counter = 0;
  for (int i = 0; i < n; i++) {
    counter += dict.get(i);
  }
  return counter;
}

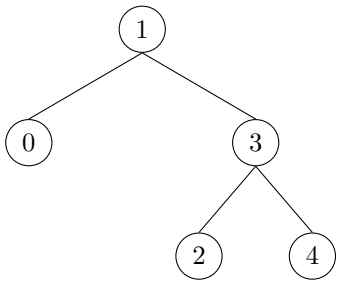
private void populate(int k, IDictionary<Integer, Integer> dict) {
  if (k == 0) {
    dict.put(0, k);
  } else {
    for (int i = 0; i < k; i++) {
      dict.put(i, i);
    }
    populate(k / 2, dict);
  }
}
```

   Nehmen Sie an, dass `put` und `get` in einem AVL-Baum mit m gespeicherten Schlüsseln jeweils eine Laufzeit von $\Theta(\log m)$ besitzen.

   Schreiben Sie eine mathematische Funktion, d.h. die Laufzeitfunktion, die die Worst-Case-Laufzeit von `test` darstellt. Am besten schreiben Sie zwei Laufzeitfunktionen auf, eine für die Laufzeit von `test` und eine für die Laufzeit von `populate`.

6. Fügen Sie nacheinander die Schlüssel 8, 7, 6 und 5 in den gegebenen AVL-Baum ein. Stellen Sie nach jedem Einfügen die AVL-Eigenschaft durch geeignete Rotationen wieder her.

   Zeichnen Sie den resultierenden AVL-Baum. Zwischenschritte können Sie zur Verdeutlichung angeben; die durchgeführten Rotationen müssen Sie nicht benennen.



7. Geben Sie für jede der folgenden Operationen eine asymptotische obere Schranke für die Worst-Case-Laufzeit in Abhängigkeit von der Anzahl n der Elemente an.
   (a) Einfügen eines Schlüssels und Suchen nach einem Schlüssel in einem binären Suchbaum.
   (b) Einfügen und Suchen in einem AVL-Baum.
   (c) Suche nach dem kleinsten Schlüssel in einem AVL-Baum mit n Elementen.
   (d) Suche nach dem k-größten Schlüssel in einem AVL-Baum mit n Elementen.

8. Vergleich von AVL-Bäumen, binären Suchbäumen und Hashtabellen
   (a) Welche Anforderungen müssen die Schlüssel eines AVL-Baums erfüllen? Welche Anforderungen gelten für die gespeicherten Werte?
   (b) Nennen Sie Situationen, in denen ein AVL-Baum gegenüber einer Hashtabelle Vorteile bietet.
   (c) Wann kann die Verwendung eines binären Suchbaums gegenüber einem AVL-Baum sinnvoll sein?

9. Gegeben sei folgende Funktion. Der Parameter `node` bezeichnet einen Knoten eines AVL-Baums.

```
function findMe(node):
  while node has a right child:
    node = node.rightChild

  return node
```

   (a) Beschreiben Sie in ein bis zwei Sätzen, was die Funktion `findMe` zurückgibt.
   (b) Wie verhält sich der Schlüssel des zurückgegebenen Knotens zum Schlüssel des als Argument übergebenen Knotens?
   (c) Welche zusätzliche Aussage können Sie treffen, wenn `node` die Wurzel des AVL-Baums ist?

10. Ein Messsystem erfasst Messwerte zusammen mit einem ganzzahligen Zeitstempel. Für jeden Zeitpunkt wird genau ein Messwert gespeichert. Das System soll folgende Operationen unterstützen:
    - Speichern eines Messwerts zu einem gegebenen Zeitpunkt,
    - Abfragen des Messwerts zu einem bestimmten Zeitpunkt,
    - Abfragen aller Messwerte innerhalb eines gegebenen Zeitintervalls $[t_1, t_2]$.

    Es werden sehr viele Messwerte gespeichert und die genannten Operationen werden regelmäßig ausgeführt.
    (a) Wählen Sie einen geeigneten abstrakten Datentyp (ADT) und eine geeignete Datenstruktur für diese Anwendung. Begründen Sie Ihre Wahl.
    (b) Beschreiben Sie, wie ein einzelner Messwert in der gewählten Datenstruktur gespeichert wird. Welche Information dient als Schlüssel und welche als Wert?
    (c) Bestimmen Sie die asymptotische Laufzeit einer Abfrage nach dem Messwert zu einem bestimmten Zeitpunkt.
    (d) Bestimmen Sie die asymptotische Laufzeit einer Abfrage nach allen Messwerten innerhalb eines Zeitintervalls $[t_1, t_2]$. Bezeichnen Sie die Anzahl der im Intervall liegenden Messwerte mit k.

11. Gegeben seien die folgenden Wörter:

    `auto`, `autobahn`, `automat`, `haus`, `haube`, `hase`.

    (a) Bauen Sie aus diesen Wörtern einen Trie auf. Zeichnen Sie den resultierenden Trie und kennzeichnen Sie, welche Knoten das Ende eines vollständigen Wortes markieren.
    (b) Welche der folgenden Zeichenfolgen sind im aufgebauten Trie als vollständige Wörter enthalten?

    `auto`, `aut`, `automat`, `haus`, `hau`, `hase`, `haube`.

    (c) Welche gemeinsamen Präfixe werden durch die Struktur des Tries besonders effizient repräsentiert?

12. Eine Suchmaschine soll eine Autovervollständigung für Suchanfragen anbieten. Während eine Benutzerin bzw. ein Benutzer eine Suchanfrage eingibt, sollen möglichst schnell alle gespeicherten Suchbegriffe angezeigt werden, die mit der bisher eingegebenen Zeichenfolge beginnen.

    Beispielsweise sind folgende Suchbegriffe gespeichert:

`algorithmus`	`algorithmen`	`algorithmik`	`algebra`
`auto`	`autobahn`	`automat`	`automatisierung`
`haus`	`haushalt`	`haustier`	`hase`

    Die Suchbegriffe werden sehr häufig durchsucht, aber nur selten verändert.

    (a) Wählen Sie einen geeigneten abstrakten Datentyp (ADT) und eine geeignete Datenstruktur für diese Anwendung. Begründen Sie Ihre Wahl anhand der benötigten Operationen.

    (b) Beschreiben Sie, wie die oben angegebenen Suchbegriffe in Ihrer gewählten Datenstruktur gespeichert werden.

    (c) Ein Benutzer gibt die Zeichenfolge `auto` ein. Welche gespeicherten Suchbegriffe müssen als mögliche Vervollständigungen gefunden werden?

    (d) Erläutern Sie, warum die von Ihnen gewählte Datenstruktur für diese Anwendung besonders geeignet ist.

    (e) Vergleichen Sie Ihre Lösung kurz mit einer Hashtabelle und einem AVL-Baum. Gehen Sie insbesondere auf die Suche nach allen Begriffen mit einem gegebenen Präfix ein.

    (f) Nennen Sie eine Situation, in der für die Speicherung derselben Suchbegriffe eine Hashtabelle gegenüber einem Trie die geeignetere Datenstruktur wäre.

Dieses Übungsblatt ist Teil der Lehrveranstaltung *„Algorithmik“*.